

المدرسة الوطنية للعلوم التطبيقية - بني ملال
ⵜⵉⵎⵙⵙⵓⵜ ⵜⵏⵎⵓⵔⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵓⵜⵉⵏ ⵜⵉⵔⵉⵔⵉⵜ - ⴰⵣⵜ ⴱⵏⵉ ⵎⵍⵍⴰⵍ
Ecole Nationale des Sciences Appliquées - Béni Mellal

École Nationale des Sciences Appliquées de Béni Mellal

ENSA Béni Mellal

RAPPORT DE MINI-PROJET

Surveillance et maintenance prédictive d'une pompe centrifuge par un système IIoT sécurisé

Prototype entièrement simulé : ESP32 · MQTT/TLS · Node-RED · Python · Segmentation IT/OT

Module	IIOT (Industrial Internet of Things)
Filière	Transformation Digitale Industrielle (TDI)
Réalisé par	**MOHAMED ELYAZID ATTARCA**
Encadré par	**Prof. Touil**
Année universitaire	2026/2027

Résumé

Ce mini-projet conçoit un système **IIoT complet et entièrement simulé** qui surveille le palier d'une pompe centrifuge afin d'anticiper la dégradation de ses roulements, première cause de panne des machines tournantes.

> **Problématique :** détecter la dégradation assez tôt pour planifier l'intervention, sans matériel physique, tout en garantissant la mise en sécurité de la pompe même sans réseau.
> **Objectif :** réaliser une chaîne capteur → réseau → supervision → décision, fiable et sécurisée, conforme aux exigences M1 à M4 du sujet.
> **Méthodologie :** 17 étapes en 5 phases ; chaque étape est réalisée, testée, prouvée par capture, documentée et versionnée sous Git.
> **Résultats :** ESP32 simulé (DS18B20, MPU6050) avec coupure locale ; MQTT chiffré TLS via Mosquitto (comptes, ACL) ; tableau de bord Node-RED avec alertes, acquittement et journal ; script Python de détection d'anomalies ; **segmentation IT/OT** documentée ; 7 tests de robustesse validés.

Mots-clés : IIoT, maintenance prédictive, ESP32, MQTT, TLS, Node-RED, segmentation IT/OT, modèle Purdue, IEC 62443, z-score.

Sommaire

Résumé et sommaire 2
1. Introduction générale 3
2. Problématique et objectifs 4
3. Analyse et méthodologie 5
4. Conception et architecture de la solution 6
5. Réalisation technique 7
5.1 Carte ESP32 · 5.2 Broker sécurisé 7
5.3 Node-RED · 5.4 Python · 5.5 Difficultés et solutions 8
6. Résultats et points forts 9
7. Limites et perspectives 10
8. Du prototype simulé à l'usine réelle : segmentation IT/OT 11
9. Conclusion générale et références 12

1. Introduction générale

1.1 Contexte

Dans l'industrie 4.0, les équipements de production sont de plus en plus connectés afin d'être surveillés en continu. Notre sujet place l'étude dans une **PME industrielle** qui exploite une **pompe centrifuge** sur sa ligne de production. Cet équipement est critique : son arrêt imprévu interrompt toute la production.

Les **roulements du palier**, qui supportent l'arbre en rotation permanente, constituent sa principale cause de défaillance. Leur usure est progressive : elle s'accompagne d'un échauffement et d'une hausse des vibrations bien avant la casse.

1.2 Intérêt de l'IIoT pour la maintenance

L'Internet Industriel des Objets (IIoT) relie des capteurs à un système de supervision par un protocole de communication machine à machine (M2M). Il permet de passer d'une maintenance subie à une maintenance fondée sur l'état réel de la machine.

Tableau 1 – *Trois stratégies de maintenance*

Stratégie	Principe	Limite
Corrective	On répare après la panne	Arrêt d'urgence, dégâts en cascade (arbre, garniture)
Préventive	On remplace selon un calendrier	Pièces changées trop tôt ou trop tard
Prédictive (notre choix)	On mesure l'état et on suit sa tendance	Nécessite capteurs, communication et analyse

Le système doit donc mesurer, transmettre, afficher, décider et tracer : c'est exactement la chaîne qu'enseigne le module IIoT.

1.3 Démarche générale

Le projet est **entièrement simulé** : aucun matériel physique n'est fourni. Nous avons découpé le travail en **17 étapes regroupées en 5 phases**. Chaque étape suit le même cycle : réaliser, tester, prouver par une capture d'écran, documenter puis enregistrer dans le dépôt Git.

Tableau 2 – *Organisation du projet en phases*

Phase	Étapes	Contenu
1. Comprendre et concevoir	1 à 3	Cahier des charges, architecture en couches, topics MQTT et QoS
2. Infrastructure	4 à 5	Environnement, dépôt Git, broker Mosquitto sécurisé
3. Carte ESP32	6 à 9	Capteurs, actionneurs, MQTT TLS, fiabilité, validation des données
4. Supervision et décision	10 à 13	Dashboard, alertes, journal, **segmentation IT/OT**
5. Valider et livrer	14 à 17	Tests de robustesse, script Python, rapport, soutenance

1.4 Organisation du rapport

Après la problématique et la méthodologie (chapitres 2 et 3), nous présentons l'architecture, la réalisation technique, les résultats des tests puis les limites (4 à 7). Le chapitre 8 traduit ensuite le prototype simulé dans une **usine réelle**, sous forme de schéma (**segmentation IT/OT**), avant la conclusion (9).

2. Problématique et objectifs

2.1 Problème étudié

Un roulement qui se dégrade (usure, perte de lubrification, désalignement, balourd) provoque un **frottement anormal**. L'énergie mécanique se transforme alors en **chaleur** et en **vibrations**. Ces deux grandeurs proviennent du même phénomène physique et sont complémentaires : **la vibration anticipe la dégradation, la température confirme sa gravité**.

Sans surveillance, la panne est découverte trop tard : une pièce de faible valeur entraîne un arrêt d'urgence et des dégâts coûteux sur l'arbre et la garniture.

2.2 Problématique

> *Comment concevoir, sans matériel physique, une chaîne IIoT fiable et sécurisée qui détecte la dégradation d'un roulement assez tôt pour planifier l'intervention, tout en garantissant la mise en sécurité de la pompe même en cas de panne du réseau ?*

2.3 Justification du choix

- **Équipement critique** : son arrêt bloque la ligne de production.
- **Mode de défaillance lent**, donc prédictible : c'est la condition de la maintenance prédictive.
- **Périmètre précis** : les défauts électriques ou hydrauliques (capteur de courant, de pression) sont volontairement exclus.

2.4 Objectifs

Objectif général : réaliser un prototype IIoT complet, du capteur virtuel au tableau de bord, qui prévient l'opérateur puis met la machine en sécurité.

Tableau 3 – *Objectifs spécifiques (modules du sujet)*

Module	Objectif spécifique
M1 – Perception	Mesurer température et vibration, valider les mesures, couper le moteur au seuil critique sans réseau
M2 – Réseau	Publier en MQTT sur des topics structurés, recevoir les commandes, justifier la QoS
M3 – Edge / décision	Tableau de bord temps réel, seuil d'alerte lissé, buzzer automatique, historique
M4 – Sécurité, fiabilité	Authentification, ACL, TLS, reconnexion, Last Will, journal horodaté, **segmentation IT/OT**
Bonus	Script Python de détection d'anomalies et d'estimation de tendance

2.5 Cahier des charges et résultats attendus

Le cahier des charges, rédigé au début du projet, fixe le besoin. Le résultat attendu est une réponse **à deux niveaux** : prévenir d'abord (buzzer), couper ensuite (relais).

Tableau 4 – *Synthèse du cahier des charges*

Élément	Spécification
Équipement surveillé	Pompe centrifuge, au niveau du palier à roulements
Mode de défaillance	Dégradation progressive des roulements : usure, perte de lubrification
Grandeurs et capteurs	Température du palier (DS18B20) ; vibration RMS (MPU6050)
Actionneurs	Buzzer : alerte (Node-RED) ; relais : coupure du moteur (ESP32)
Seuils (alerte · critique)	Température 75 °C · 85 °C (normal 40 à 70 °C) ; vibration 0,5 g · 1,0 g (normal < 0,3 g)
Plages valides	0 à 120 °C et 0 à 8 g ; NaN et −127 °C rejetés
Contraintes	Simulation Wokwi (VS Code + PlatformIO), MQTT TLS port 8883, Mosquitto local, Node-RED, Python (bonus)

3. Analyse et méthodologie

3.1 Analyse du besoin et choix des capteurs

La première étape a été la rédaction d'un **cahier des charges** : équipement, mode de défaillance, grandeurs, capteurs, seuils et plages de validité.

Tableau 5 – *Grandeurs mesurées et capteurs*

Grandeur	Capteur	Justification
Température du palier	DS18B20 (−55 à +125 °C)	Sonde de contact qui mesure le métal ; le DHT22 mesure l'air et plafonne à 80 °C
Vibration	MPU6050 (±8 g)	Accéléromètre 3 axes, posé à plat sur le palier

Calcul de la vibration. La gravité est retirée sur l'axe vertical : $v = \sqrt{(x^2 + y^2 + (z - 1)^2)}$. L'ESP32 effectue 100 lectures en 2 s et publie leur **valeur efficace (RMS)**. La formule $\sqrt{(x^2 + y^2 + z^2)} - 1$ a été écartée car la gravité y écrase les vibrations horizontales.

3.2 Répartition de la décision

- **Seuil d'alerte au niveau edge (Node-RED)** : il profite du lissage, de l'affichage et de la traçabilité.
- **Seuil critique au niveau device (ESP32)** : la coupure ne dépend ni du réseau, ni du broker, ni du serveur.
- **Sécurité positive** : le relais doit être alimenté pour que le moteur tourne ; si l'ESP32 plante, la pompe s'arrête.
- **Réarmement manuel** sur place uniquement, avec des mesures valides et sous les seuils d'alerte.

3.3 Démarche adoptée

Nous avons suivi une démarche **incrémentale** : chaque couche est construite puis testée avant la suivante (broker, puis carte, puis supervision). Chaque erreur rencontrée a été analysée et documentée. Cette démarche a permis de corriger tôt des défauts réels, par exemple un réarmement accepté avec un capteur débranché.

3.4 Justification des choix techniques

Tableau 6 – *Outils et technologies retenus*

Choix	Justification
Wokwi dans VS Code + PlatformIO	Simulation et compilation en local : plus de dépendance aux serveurs de Wokwi.com ; l'ESP32 joint le broker du PC sans tunnel
ESP32	Wi-Fi intégré, deux cœurs, assez puissant pour la RMS et la décision locale
MQTT	Protocole léger, publish/subscribe adapté à des capteurs qui poussent leurs mesures
Mosquitto en local	Broker open source dont nous configurons nous-mêmes la sécurité (objectif pédagogique)
Node-RED + Dashboard 2.0	Décision, tableau de bord et client MQTT natif dans un seul outil
Python (NumPy, matplotlib)	Analyse de l'historique, indépendante de Node-RED

4. Conception et architecture de la solution

4.1 Architecture en quatre couches

Tableau 7 – *Placement des composants dans les couches du sujet*

Couche	Composants
M1 – Perception	DS18B20, MPU6050, ESP32 (lecture, validation, RMS, seuil critique), relais, buzzer, bouton de réarmement
M2 – Réseau	Wi-Fi simulé, passerelle Wokwi (host.wokwi.internal), broker Mosquitto, messages JSON
M3 – Edge / décision	Node-RED : validation, moyenne glissante, seuil d'alerte, commande du buzzer, dashboard, journal ; script Python
M4 – Sécurité (transversale)	Comptes, ACL, TLS, reconnexion, Last Will, validation des données, journal, secrets hors du dépôt

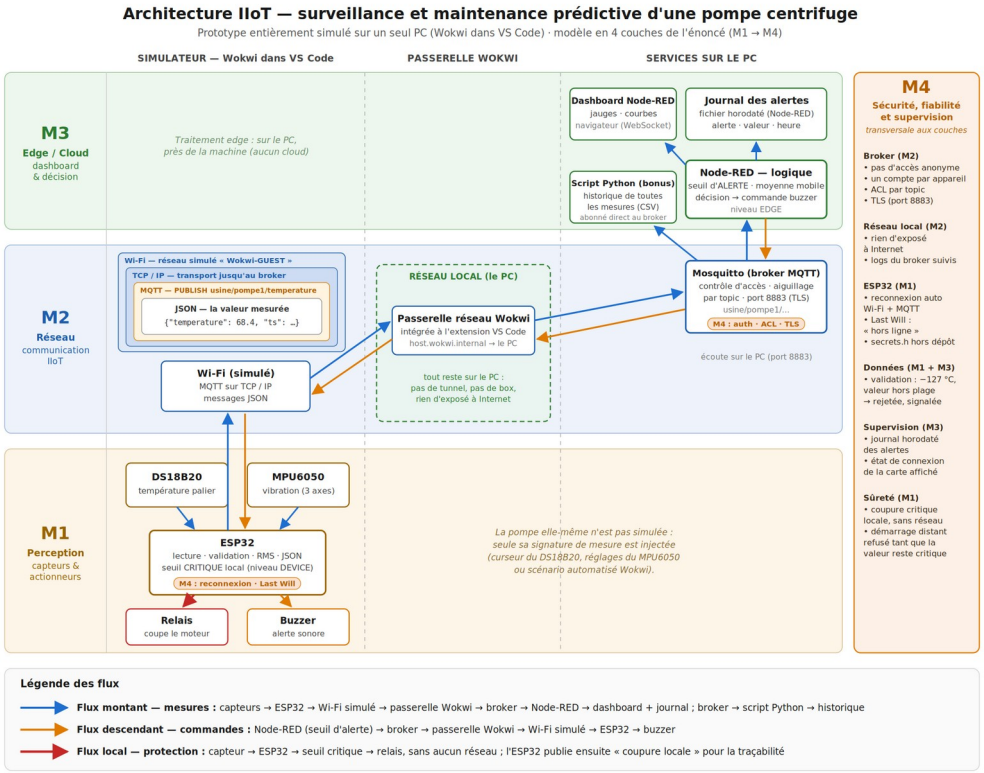


Figure 1 – *Architecture générale du prototype et circulation des messages*

4.2 Flux et protocole MQTT

- **Flux montant :** capteurs → ESP32 → broker → Node-RED (dashboard, journal) et, en parallèle, → Python.
- **Flux descendant :** Node-RED → broker → ESP32 → buzzer.
- **Flux local :** capteur → ESP32 → relais, sans réseau ; la coupure est publiée ensuite pour la traçabilité.

Les topics suivent l'arborescence **usine/ligne1/pompe1/…** ; chaque mesure est un JSON {valeur, unite, qualite, ts}. **QoS 0** pour les mesures (une nouvelle arrive toutes les 2 s), **QoS 1** pour les commandes, alertes et états. Seuls les **états** sont retenus par le broker : un ordre retenu serait rejoué à tort.

5. Réalisation technique

5.1 Carte ESP32 (firmware C++)

Le firmware répartit le travail sur les **deux cœurs** de l'ESP32 : la **sécurité** (mesures, validation, seuil critique, relais) tourne sur le cœur 1 et le **réseau** (Wi-Fi, TLS, MQTT) sur le cœur 0. Les deux tâches échangent par une zone partagée protégée par un **mutex**. Ce choix corrige un défaut constaté : la poignée de main TLS bloquait la coupure pendant environ 6 s.

Tableau 8 – *Règles de validation des mesures dans l'ESP32*

Règle	Pourquoi	Résultat du test
Plage plausible (0–120 °C, 0–8 g)	Capteur débranché ou en court-circuit	−127 °C → « erreur »
Valeur piège 85,0 °C rejetée	Renvoyée par un DS18B20 après une erreur d'alimentation	Pas de coupure ; 85,1 °C → coupure
Saut > 10 °C en 2 s ignoré	Un parasite ne doit pas arrêter la pompe	Ignoré, puis confirmé 2 s plus tard
MPU6050 muet (bus I2C, gravité < 0,2 g)	Débranché, il simulerait 1 g de vibration	valeur null, pas de coupure
Réarmement avec mesures valides	Faille trouvée : −127 °C passait « < 75 °C »	Réarmement refusé

Fiabilité de la liaison : keep-alive de 30 s, **Last Will** « hors_ligne » publié par le broker après 45 s de silence, reconnexion automatique toutes les 5 s et renvoi des états. Côté Node-RED, un chien de garde de 10 s **anticipe** la perte de la carte ; le Last Will la **confirme**.

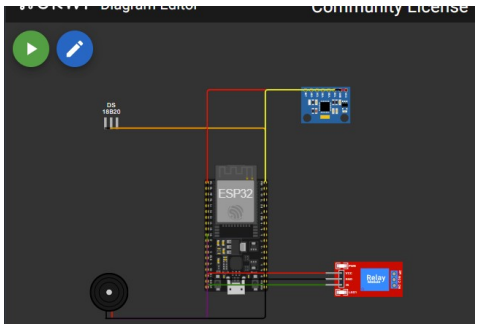


Figure 2 – *Simulateur Wokwi : ESP32, DS18B20, MPU6050, relais et buzzer*

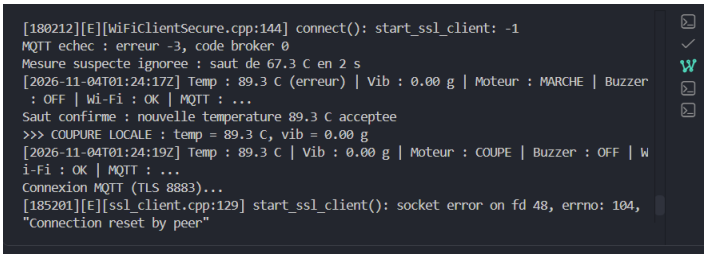


Figure 3 – *Moniteur série : coupure locale à 89,3 °C pendant l'arrêt du broker (test T5)*

5.2 Broker Mosquitto sécurisé

Le broker n'accepte que **TLS sur le port 8883** (port 1883 fermé), avec une autorité de certification locale créée avec OpenSSL. L'échange de clé ECDHE et le chiffrement AES-256-GCM protègent les données ; l'ESP32 négocie TLS 1.2, Node-RED et Python TLS 1.3.

Tableau 9 – *Protections du broker*

Protection	Contre quoi
Pas d'accès anonyme, un compte par appareil (esp32, nodered, python)	Connexion d'un inconnu, usurpation
ACL par topic (python en lecture seule)	Compte volé qui envoie de fausses commandes
TLS 8883 uniquement	Interception des identifiants et des données
Journaux du broker	Détection des tentatives de connexion
Mots de passe hors du code et de Git (secrets.h exclu)	Fuite des identifiants via le dépôt

5.3 Supervision et décision dans Node-RED

Chaque message reçu est d'abord **validé** (7 règles : JSON, grandeur, unité, qualité, horodatage, bornes). La décision suit ensuite une chaîne simple :

- **Moyenne glissante** sur 5 mesures (10 s) : un pic isolé ne déclenche pas d'alerte.
- **Seuil avec hystérésis** : l'alerte commence au-dessus de 75 °C et ne finit que sous 72 °C (0,5 g / 0,4 g), sans clignotement.
- **Alertes numérotées** (un id par alerte) publiées sur le topic alertes, seule source de vérité : la liste, le buzzer et le journal voient les mêmes événements.
- **Acquittement** depuis le tableau de bord, transmis par le broker ; une alerte ne sort de la liste qu'une fois acquittée **et** terminée.
- **Règle des 3 valeurs** : trois mesures en erreur de suite déclenchent une alerte « défaut capteur ».

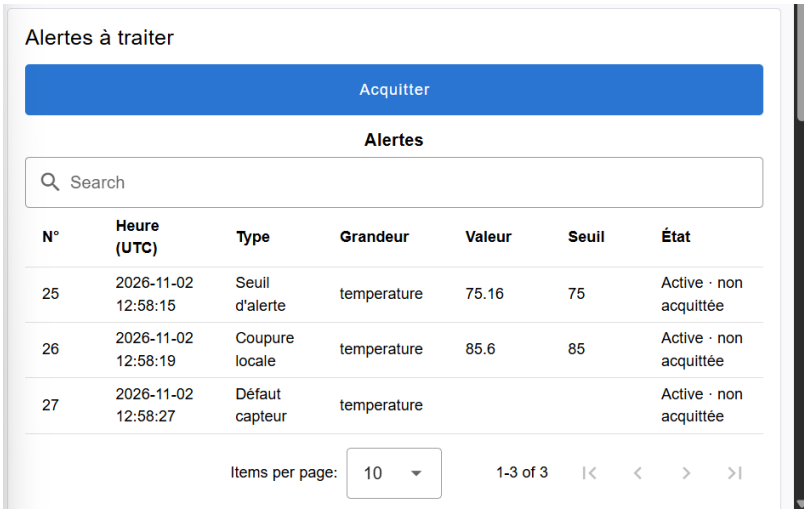


Figure 4 – *Liste des alertes à traiter : seuil d'alerte, coupure locale et défaut capteur*

Le **journal horodaté** conserve tous les événements sans jamais les effacer. Chaque ligne porte deux heures (réception par Node-RED et mesure par l'ESP32). Un filtre ignore les états rejoués à la reconnexion, et une **session MQTT persistante** permet de rattraper les événements survenus pendant une absence de Node-RED.

5.4 Analyse Python (bonus)

Un **collecteur indépendant** de Node-RED (compte en lecture seule) enregistre son propre historique. L'analyse repère les anomalies par **z-score robuste** : z = (mesure − médiane) / (1,4826 × MAD), calculé sur la minute précédente ; la médiane et le MAD ne sont pas faussés par un saut isolé. Une **droite des moindres carrés** estime ensuite le temps restant avant 75 °C et 85 °C, uniquement si 15 minutes de données sont disponibles.

5.5 Difficultés rencontrées et solutions

Tableau 10 – *Principales difficultés et solutions apportées*

Difficulté	Cause identifiée	Solution
Aucune mesure publiée	Heure NTP non reçue via Wokwi	Trois serveurs NTP
Coupure retardée d'environ 6 s	Poignée de main TLS bloquante	Deux cœurs + mutex
Événements perdus (Node-RED arrêté)	Session MQTT non persistante	Session gardée, sans désabonnement
Anomalies non détectées (Python)	Référence faussée par un saut	Fenêtre en temps + médiane/MAD
Prévision « alerte dans 2 min »	Droite calculée sur 78 s	Prévision après 15 min de données

6. Résultats et points forts

6.1 Tests fonctionnels et de robustesse

Tableau 11 – *Résultats des tests (étape 14)*

Test	Scénario	Résultat obtenu
T1	Chaîne complète : seuil, alerte, buzzer, coupure, acquittement, réarmement	✓ 3 alertes traitées, liste vide à la fin
T2	Connexion anonyme puis mauvais mot de passe	✓ « not authorised » dans le log
T3	Compte python tente de publier une commande	✓ Lecture autorisée, publication refusée
T4	Arrêt brutal de la simulation	✓ Chien de garde (10 s) puis Last Will
T5	Redémarrage du broker	✓ Coupure locale à 89,3 °C sans broker, reconnexion seule
T6	Valeurs aberrantes (85,0 °C, saut, capteurs débranchés)	✓ « erreur », aucune fausse coupure
T7	Node-RED arrêté	✓ Sécurité intacte, événements rattrapés

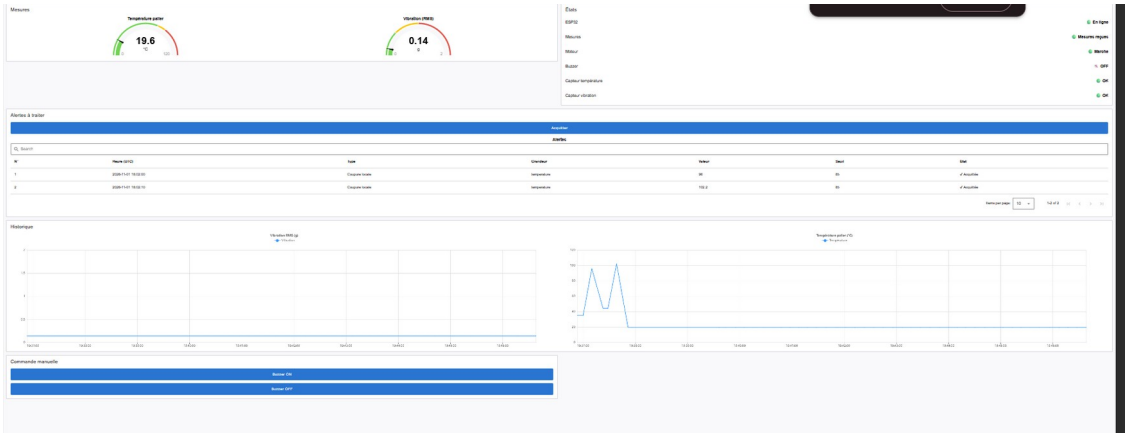


Figure 5 – *Tableau de bord Node-RED final : jauges, voyants d'état, courbes et alertes*

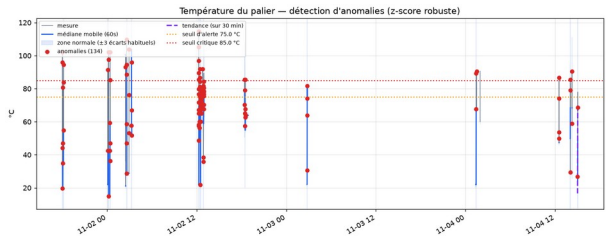


Figure 6 – *Analyse Python des vraies mesures (01/11 → 04/11) : 134 sauts détectés, aucune prévision (moins de 15 min de données récentes)*

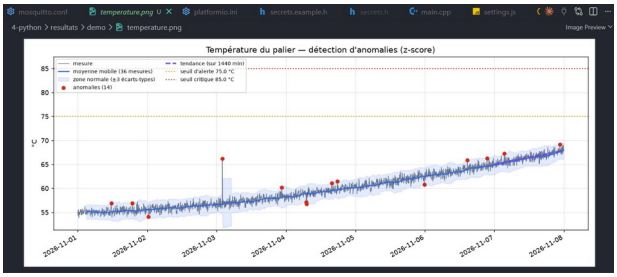
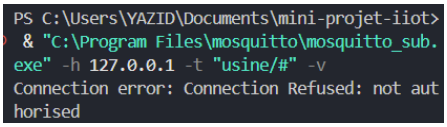


Figure 7 – *Analyse Python sur une usure simulée de 7 jours : anomalies (points rouges) et tendance*

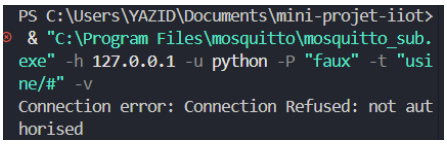
6.2 Comparaison avec les objectifs et valeur ajoutée

Les exigences M1 à M4 et le bonus Python sont **réalisés et prouvés** ; la segmentation est décrite au chapitre 8. Sur l'usure simulée, le script annonce l'**alerte dans 2,3 jours** et le seuil critique dans 5,8 jours, alors que Node-RED ne signale encore rien : c'est l'apport de la maintenance prédictive. Points forts : **sécurité indépendante du réseau**, défense en profondeur (validation dans l'ESP32 puis dans Node-RED), traçabilité complète et sécurité appliquée de bout en bout.

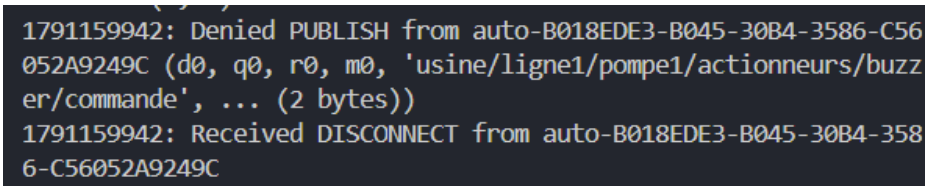
6.3 Tests de sécurité : preuves



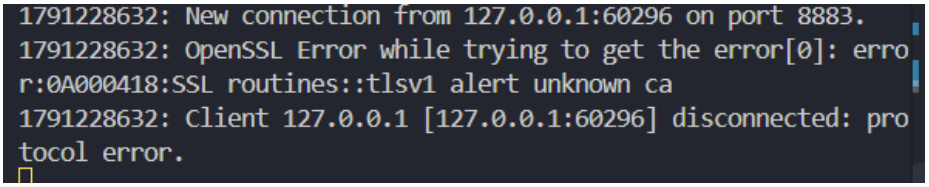
(a) Connexion anonyme refusée (T2)



(b) Mauvais mot de passe refusé (T2)



(c) ACL : publication refusée au compte python (T3)



(d) Certificat d'une CA inconnue refusé (TLS)

Figure 8 – *Tests de sécurité du broker Mosquitto : authentification, ACL et TLS*

7. Limites et perspectives

7.1 Limites identifiées

Tableau 12 – *Limites du prototype en bref (elles viennent surtout de la simulation)*

Domaine	Limite	En bref
Simulation	Wokwi lent (9 à 46 % du temps réel), Wi-Fi impossible à couper	Délais allongés ; perte réseau testée en arrêtant le broker
Démarrage	Heure NTP absente les premières secondes	Supervision aveugle un instant ; la sécurité locale reste active
Données	Historique CSV, persistance du broker non activée	Fichiers sans limite ; messages en attente perdus si le broker redémarre
Prévision	Modèle linéaire, testé sur une usure simulée	Une usure qui s'accélère serait sous-estimée
Choix de sécurité	Un capteur en défaut ne coupe pas la pompe	L'autre grandeur protège seule
Déploiement	Un seul PC, sans réseau d'usine	Déploiement réel décrit au chapitre 8

7.2 Améliorations possibles

- Base de données de séries temporelles (InfluxDB) et visualisation longue durée.
- Persistance Mosquitto et **broker local en OT relié par un pont** au broker de la DMZ.
- Coupure de repli si un capteur reste en défaut plus de 30 s.
- Modèle de prévision non linéaire ou apprentissage automatique sur un historique réel.

7.3 Difficultés restantes

- **Ordre de reconnexion** : une alerte peut échapper à Node-RED s'il revient après l'ESP32.

7.4 Perspectives d'évolution et applications

L'étape suivante serait un **banc réel** avec un ESP32 et des capteurs physiques, puis l'ajout de grandeurs (courant moteur, pression) pour couvrir les défauts électriques et hydrauliques, aujourd'hui hors périmètre. L'arborescence **usine/ligne/pompe** permet d'ajouter d'autres machines sans changer l'architecture : la solution s'applique à toute **machine tournante** (compresseurs, ventilateurs, moteurs) et à une flotte d'équipements supervisée depuis une salle de contrôle.

8. Du prototype simulé à l'usine réelle : segmentation IT/OT

Tout notre travail a été réalisé **en simulation, sur un seul PC** : l'ESP32 Wokwi, Mosquitto, Node-RED et Python partagent la même machine. Le sujet demande de décrire comment ce système serait isolé dans une vraie usine. Nous avons donc **traduit le prototype en déploiement réel**, sous forme de **schéma uniquement** : rien n'a été configuré, mais chaque choix s'appuie sur ce que nous avons construit et testé.

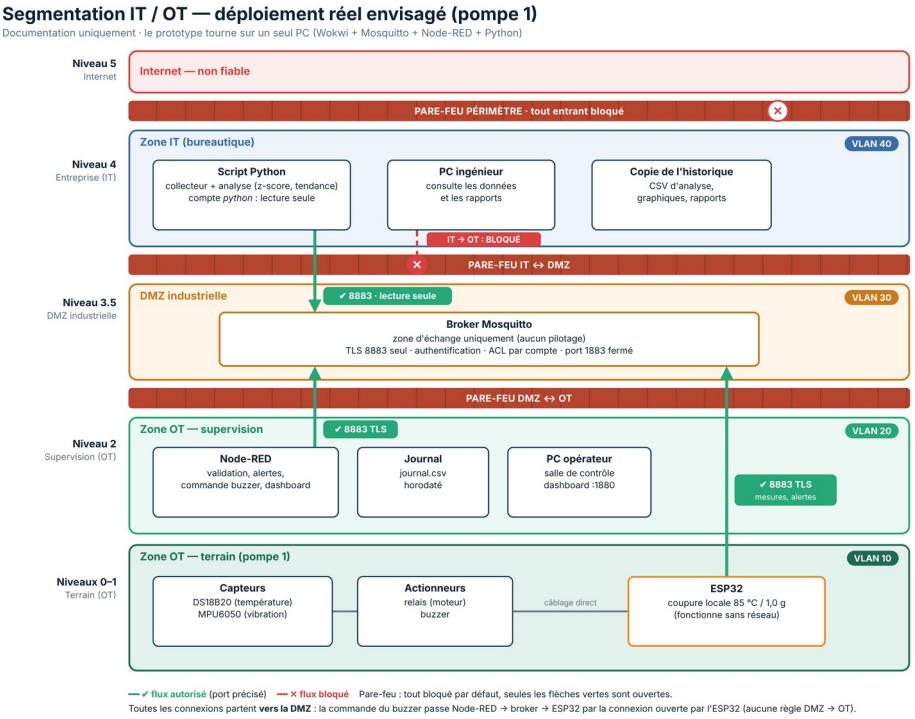


Figure 9 – *Déploiement réel envisagé : zones, VLAN, pare-feux et flux autorisés*

8.1 Ce que nous avons fait

- Nous avons classé chaque élément du prototype par **niveau du modèle Purdue** vu en cours : terrain (ESP32, capteurs, actionneurs), supervision (Node-RED, journal, PC opérateur), **DMZ** (broker) et IT (Python, PC ingénieur).
- Nous avons donné à chaque zone un **VLAN** (10, 20, 30, 40) et placé un **pare-feu** entre les zones, qui **bloque tout par défaut**.
- Nous n'avons ouvert que **MQTT 8883 (TLS) vers la DMZ** : l'IT ne parle jamais directement aux machines, et Internet n'entre nulle part.

8.2 Pourquoi ces choix

Tableau 13 – *Choix retenus et justification*

Choix retenu	Raison
Broker en DMZ	Le broker est le point d'échange entre l'OT et l'IT. En DMZ, chacun s'y connecte sans entrer dans la zone de l'autre. S'il tombe, l'ESP32 coupe quand même la pompe (test T5).
Node-RED en OT (supervision)	Node-RED ne fait pas qu'afficher : il décide des alertes et commande le buzzer. Un outil qui pilote une machine reste en OT, avec le PC opérateur de la salle de contrôle.
Python en IT	Le script lit et analyse seulement l'historique. Son compte MQTT est en lecture seule : même compromis, il ne peut envoyer aucune commande (test T3).
Connexions vers la DMZ	Aucune connexion ne part de la DMZ vers l'OT. La commande du buzzer arrive à l'ESP32 par la connexion que la carte a elle-même ouverte vers le broker.
VLAN + pare-feu	Un VLAN sépare les réseaux mais ne contrôle pas ce qui passe entre eux. Le pare-feu bloque tout par défaut et n'ouvre que le port 8883 vers la DMZ.

8.3 Les 7 exigences de la norme IEC 62443 en bref

La norme **IEC 62443** organise le réseau en **zones** (nos VLAN) reliées par des **conduits** contrôlés (nos flux 8883 TLS). Elle fixe 7 exigences fondamentales :

Tableau 14 – *Exigences fondamentales IEC 62443 et mise en œuvre dans le projet*

N°	Exigence	Dans notre projet
FR1	Identifier et authentifier	Comptes MQTT avec mot de passe, accès anonyme refusé (T2)
FR2	Contrôler l'usage	ACL par compte : python en lecture seule (T3)
FR3	Intégrité du système	TLS, validation des mesures dans l'ESP32 et Node-RED
FR4	Confidentialité des données	Chiffrement TLS (AES-256-GCM)
FR5	Flux de données restreints	VLAN + pare-feux, IT → OT bloqué
FR6	Réagir aux événements	Journal horodaté, alertes et acquittement
FR7	Disponibilité	Coupure locale sans réseau (T5), reconnexion automatique

9. Conclusion générale et références

9.1 Bilan

Ce mini-projet a abouti à une chaîne IIoT complète et entièrement simulée : mesure et validation sur l'ESP32, communication MQTT chiffrée, supervision et décision dans Node-RED, traçabilité, analyse prédictive en Python et segmentation IT/OT du déploiement réel. Le principe directeur a été respecté : **prévenir d'abord, couper ensuite, et ne jamais faire dépendre la sécurité du réseau**.

9.2 Compétences acquises

- Programmation embarquée multitâche (FreeRTOS, deux cœurs, mutex) et traitement du signal (RMS).
- Protocole MQTT : topics, QoS, messages retenus, Last Will, sessions persistantes.
- Cybersécurité industrielle : TLS, ACL, moindre privilège, modèle Purdue, IEC 62443.
- Supervision Node-RED, analyse de données (NumPy) et méthode : tests, preuves, versionnement Git.

9.3 Apports du module IIoT

Le module nous a fourni le cadre du projet : l'architecture en couches (perception, réseau, edge, sécurité), la communication M2M par MQTT et les notions de sécurité industrielle (modèle Purdue, zones et conduits). Il nous a surtout montré qu'un système industriel connecté se juge autant à sa **fiabilité** et à sa **sécurité** qu'à ses fonctionnalités.

Vidéo de démonstration : https://youtu.be/dGGZYHPi2UU (9 min 49)

Dépôt du projet : https://github.com/yaziat/mini-projet-iiot

Références

[1] Sujet du mini-projet et cours du module IIOT, Prof. Touil, ENSA Béni Mellal, 2026/2027.
[2] OASIS, MQTT Version 3.1.1 et MQTT Version 5.0, spécifications du protocole.
[3] Eclipse Mosquitto, documentation officielle : mosquitto.conf, mosquitto_passwd, ACL, TLS.
[4] Node-RED et FlowFuse Dashboard 2.0, documentation officielle.
[5] Wokwi, documentation du simulateur ESP32 et de l'extension VS Code ; PlatformIO, documentation.
[6] Espressif Systems, ESP32 Technical Reference Manual ; bibliothèque arduino-mqtt (256dpi).
[7] Maxim Integrated, fiche technique DS18B20 ; InvenSense, fiche technique MPU-6000/MPU-6050.
[8] IEC 62443, Sécurité des réseaux et systèmes de contrôle industriels ; modèle Purdue (ISA-95).
[9] Documentation NumPy et Matplotlib (analyse et graphiques Python).